

A Schema-First Blueprint for Integrating Performant Fire and Soot Effects into the GameNext Research Ecosystem

The objective of this research report is to define the next directions for console and PC gaming by formally integrating fire and soot visual effects as first-class research objects within the GameNext ecosystem. The central thesis is to establish a structured framework that balances high-fidelity, believable visuals with stringent performance constraints, leveraging a schema-first, index-driven architecture to maximize usability through AI-powered tooling. This report outlines a comprehensive plan divided into two primary tracks: (1) the creation of a canonical, engine-agnostic runtime module for fire and soot mechanics, and (2) the deep integration of this new domain into the GameNext knowledgebase, database, and AI-chat pipelines. The proposed approach treats fire and soot not as isolated visual flourishes but as tightly coupled subdomains of the established grime and damage systems, designed to be navigable and editable by both human designers and artificial intelligence agents. Every aspect of this proposal is grounded in the principles of reusability, performance efficiency, and architectural consistency, aiming to create a scalable and maintainable model for future visual effect systems.

Canonical Runtime Architecture for Engine-Agnostic Effects

The foundational step in formalizing fire and soot mechanics is the creation of a canonical, engine-agnostic runtime module. This module serves as the single source of truth for the system's logic, abstracting away the complexities of specific game engines like Unreal or Unity to ensure consistent behavior and simplify long-term maintenance. The core architectural philosophy is predicated on achieving visually compelling effects at a computational cost that is effectively zero compared to modern rendering workloads, making it viable for resource-constrained platforms like consoles and high-fidelity PC titles alike [23](#) [25](#). This is accomplished through a highly constrained design that relies exclusively on scalar CPU updates, avoids expensive GPU resources, and strategically reuses existing engine systems for proximity checks and material state management.

The proposed runtime module is built around a small state structure attached to each character actor or pawn, which is updated every frame . This structure, tentatively named `FireExposureState`, contains a set of scalar fields representing the character's interaction with a fire emitter. These fields include `exposure_heat` and `exposure_smoke`, which track the intensity of thermal and smoke-based exposure over time, and per-region soot accumulations (`soot_torso`, `soot_head`, `soot_arms`, `soot_legs`) that represent the persistent visual degradation . All these values are clamped to a range of [0, 1], representing no effect to full saturation . The entire update process is managed by a simple, cheap integrator function that runs on the CPU. For instance, the next state of heat exposure is calculated as

$$exposureHeat_{next} = clamp_{01}(exposureHeat + k_{in} \cdot inInnerZone - k_{out} \cdot notInZone),$$

where `k_in` and `k_out` are coefficients for buildup and decay, and `inInnerZone` is a boolean flag indicating proximity to the fire source . This purely scalar, CPU-bound computation ensures the overhead is trivial, likely costing less than a few floating-point operations per frame per character, and scales efficiently with the number of characters in the scene 32 68 .

A critical component of this architecture is the explicit decision to decouple the immediate sensory experience of "live fire" from the lasting consequence of "residual soot" . This separation is fundamental to achieving the desired balance between believability and player comfort. The "live fire" effect is modeled as a short-lived mask, driven directly by being within the fire's radius or taking direct damage ticks. It exhibits high intensity while the character is engulfed but decays rapidly—on the order of hundreds of milliseconds—once they exit the hazardous zone . This ephemeral nature is used to drive transient visual phenomena that provide immediate feedback, such as additive emissive flicker, localized heat shimmer distortion, and brief bursts of smoke particles. In contrast, the "sootMask" is derived from the accumulated `perRegionSoot` values and persists indefinitely during normal gameplay . It has no automatic time-based decay and is only modified by explicit "cleaning" events, such as interacting with a med-center, entering a shower, or being caught in a heavy rain event. This distinction elegantly solves the common problem of soot appearing and disappearing inconsistently; the live fire effect fades quickly, but the soot—a permanent mark of the encounter—remains visible until actively removed . This creates a more coherent and intentional visual narrative.

To enforce engine-agnosticism, the core logic of the fire-soot system is encapsulated in a platform-neutral C or C++ header file . This header would declare a function like `update_fire_soot_state(FireExposureState<em>, const FireZoneSample</em>, float dt_seconds)`. This function would contain the exact integrator equations and update rules, providing a stable API that any engine can

call . By defining the system at this level, it becomes independent of Unreal's component model or Unity's MonoBehaviours, aligning with the broader GameNext strategy of treating Rust, C, GLSL, and SQL as projections of a canonical schema rather than as engine-specific truths . The engine-specific implementations then act as thin adapters. For example, an Unreal implementation would involve creating a `UFireExposureComponent` that wraps the `FireExposureState` struct and hooks into Unreal's native damage event system to sample fire zones. A Unity implementation would use a `MonoBehaviour` script that performs similar sampling and calls the core logic . This layered approach prevents code drift and duplication, ensuring that all engine versions share the same underlying behavioral contract.

Finally, the architecture is designed for seamless persistence across save/load cycles and level reloads. The small `FireExposureState` struct, containing just a handful of floats for each region and exposure type, is saved along with the character's other state information . Upon loading, the state is re-instantiated, restoring the soot levels exactly as they were before the interruption. This preserves visual continuity without requiring any expensive dynamic textures or meshes to remain resident in memory between sessions . The entire runtime model is thus a lightweight, efficient, and robust foundation upon which the richer, engine-specific visual effects and AI-driven tooling can be built.

Deep Integration with Grime Frameworks and Telemetry Systems

For the fire and soot system to become a cohesive part of the GameNext ecosystem, it must be deeply integrated with the pre-existing "grime" framework and the constellation's telemetry infrastructure. This tight coupling is a deliberate design choice aimed at maximizing reusability, maintaining visual consistency, and leveraging established data pipelines for analytics and machine learning . Rather than creating a standalone system, fire-soot is treated as a specific, specialized application of the general grime-binding pattern, extending its capabilities without introducing new, parallel mechanisms .

The integration begins with the extension of the grime binding schema, which governs how material parameters are updated based on a character's state . The canonical schema for a character's material state would be augmented with new fields specifically for fire-related effects. These additions include `soot_accumulated_regions`, a vector of scalars corresponding to the torso, head, arms, and legs; `soot_visual_scalar` (a value from 0 to 1); and a reference to a `fire_soot_profile_id` that links to a style

pack or configuration asset . The visual output is then defined by a standard formula, $soot_{visual} = clamp_{01}(soot_{base} + soot_{accumulated})$, where `soot_base` is a shader-level default and `soot_accumulated` is the value derived from the runtime state . This approach ensures that the final appearance of soot is driven by the same system that controls other forms of wear and tear like mud or dust, allowing artists to manage all grime types through a unified interface.

Shader integration is designed for maximum compatibility and minimal overhead. The system reuses existing shader parameters wherever possible. For example, the `flowDirX/Y` vectors, already present in the grime schema to bias streaking direction, can be repurposed to simulate the upward or outward flow of hot smoke and soot, eliminating the need for new texture maps or complex procedural generation in the fragment shader . The shader contract requires a few standard lerp operations to blend the soot effect into the base material. The final albedo is computed as $BaseColor' = lerp(BaseColor, SootColor, soot_{visual})$, and the roughness is adjusted with $Roughness' = lerp(BaseRough, BurntRough, soot_{visual})$. An optional normal detail overlay can also be blended using the `soot_visual` scalar to add subtle pitting or surface imperfection . This reliance on standard, well-understood shader patterns makes integration straightforward for any artist familiar with physically based rendering (PBR) pipelines.

This tight integration extends to the persistence and cleaning systems. The `perRegionSoot` state is stored as part of the character's runtime state, mirroring the way other grime states are managed . Crucially, key fields from this state—such as `soot_torso`, `soot_head`, `exposure_heat`, and `exposure_smoke`—are automatically mirrored into the existing grime binding table in the SQLite telemetry database whenever a state change occurs . This allows for powerful post-hoc analysis without requiring any special handling for fire/soot data. Cleaning mechanics, such as those found in a med-center or shower, are also formalized as extensions of the grime removal system. When a cleaning event is triggered, it can either instantly reset the soot values or apply a gradual decay curve, $soot_{next} = max(0, soot_{prev} - cleanupRate \cdot \Delta t)$. During cleaning, the `wetness` parameter is temporarily boosted, tying the visual effect of cleaning back to the existing wetness/drying pipeline . This unified approach ensures that all grime-related data—from initial accumulation to final removal—is logged consistently and can be queried together, providing a holistic view of a character's environmental interaction history.

The following table summarizes the proposed extensions to the existing grime binding schema to accommodate the fire-soot system:

Schema Field Name	Data Type	Description	Source of Truth
soot_accumulated_regions	Vector (4-6 elements)	Per-region soot accumulation values (torso, head, etc.), clamped to [0, 1]. Derived from exposure integrators.	FireExposureState runtime struct
soot_visual_scalar	float	A consolidated visual weight (0 to 1) derived from soot_accumulated_regions and base grime levels.	Formula Catalog entry
fire_soot_profile_id	string/ID	A reference to a configuration profile that defines accumulation rates, cleaning curves, and stylistic parameters.	constellationindex.db
cleaning_event	Object	A log of cleaning events, including type (med-center, rain), cleanup rate, and duration.	Custom grime_cleaning_events table

By embedding the fire-soot system within the grime framework, the research achieves several strategic goals. It reduces the cognitive load on artists and designers by building on familiar concepts. It ensures visual consistency by using the same shader parameters and UI controls. And most importantly, it connects the new system to the powerful data collection and analysis pipelines that are already in place, paving the way for quantitative validation and AI-driven optimization.

Extending the Constellation Index for AI-Navigable Knowledge

To elevate fire and soot from a simple feature to a first-class research object, it is essential to embed it within the GameNext constellation index—the central, queryable database that serves as the ecosystem's collective brain . This involves more than just adding new files; it requires a methodical extension of the index's schemas, formula catalogs, and semantic search capabilities to make the fire-soot domain as navigable and understandable to an AI agent as the existing Queryable Geometry system [50](#) . The goal is to enable natural language queries like "What are the formulas for soot accumulation?" or "Show me all instances where soot exceeded 0.5 after inner-zone exposure" with low token cost and high accuracy, moving beyond brute-force code scanning .

The first step is to extend the telemetry schema within the `constellationindex.db` to capture the relevant data points of the fire-soot system. This means augmenting the existing per-frame character state table with new columns to log the scalar values tracked by the `FireExposureState` . Specifically, columns for `soot_torso`, `soot_head`,

soot_arms, soot_legs, exposure_heat, exposure_smoke, and fire_zone_id (to identify which specific fire emitter was responsible) must be added . Furthermore, a dedicated grime_cleaning_events table should be created to log the timing, type, and parameters of all cleaning actions, providing a complete audit trail of a character's grime lifecycle . To ensure fast querying, appropriate indexes must be created on these tables, particularly composite indexes on (character_id, timestamp_ms) and (fire_zone_id, timestamp_ms), allowing the AI to efficiently filter and aggregate data .

The heart of the AI-navigable knowledge lies in the formula catalog, which is already used to store mathematical models for BCI patterns . The fire-soot system will introduce its own set of formulas that describe its core behaviors. Key entries in the catalog will include:

- **Soot Accumulation Formula:** A symbolic representation of the update rule, such as $soot_{region,next} = clamp(0, soot_{region,prev} + \alpha \cdot exposureSmoke + \beta \cdot exposureHeat)$. This formula would have associated parameters (α , β) and confidence scores.
- **Exposure Integrator Formulas:** Descriptions of the integrators for `exposureHeat` and `exposureSmoke`, detailing their build-up and decay rates (k_{in} , k_{out}) .
- **Cleaning Curves:** Mathematical descriptions of the soot removal functions, whether instantaneous or gradual ($soot_{next} = max(0, soot_{prev} - cleanupRate \cdot \Delta t)$) .

These formulas will be tagged with metadata, including their domain (fire_grime), provenance information (e.g., sourcecommit, derivedfrom), and validation status, mirroring the existing patterns for BCI formulas . This rich metadata is critical for enabling the AI to reason about the relationships between different parts of the system and understand the origin and reliability of its knowledge.

To make this indexed knowledge actionable, the AI-chat interface must be enhanced. This involves creating new, patternized documentation generation functions, analogous to an existing generatepatternreport function. A

generate_fire_soot_report(fire_profile_id) function could produce a concise markdown or JSON summary of a given fire profile, detailing its accumulation formulas, typical zone configurations, and expected visual characteristics . More powerfully, the AI can be prompted to synthesize new fire profiles by combining and modifying existing formulas from the catalog. For instance, a designer could ask, "Propose a new fire profile based on 'campfire' that has slow soot accumulation and a gentle cleaning arc when entering water," and the AI, guided by the indexed formulas, could generate a plausible configuration. This transforms the AI from a reactive lookup tool into a proactive design partner.

The following table outlines the proposed schema extensions for the `constellationindex.db` to support the fire-soot domain:

Table Name	New Columns	Purpose
<code>character_frame_state</code>	<code>soot_torso</code> (float), <code>soot_head</code> (float), <code>soot_arms</code> (float), <code>soot_legs</code> (float), <code>exposure_heat</code> (float), <code>exposure_smoke</code> (float), <code>fire_zone_id</code> (int)	Log per-frame soot levels and exposure states for all characters.
<code>grime_cleaning_events</code>	<code>event_type</code> (string), <code>cleanup_rate</code> (float), <code>duration_ms</code> (int), <code>before_soot</code> (json), <code>after_soot</code> (json)	Record and analyze all grime removal events.
<code>formulacatalog</code>	<code>domain</code> (string, e.g., "fire_grime"), <code>symbolic_formula</code> (text), <code>parameter_defaults</code> (json), <code>confidence</code> (float)	Store and version-control the mathematical models for soot accumulation and cleaning.
<code>semantic_search_embeddings</code>	<code>entity_name</code> (text), <code>embedding_vector</code> (blob), <code>source_table</code> (text)	Generate embeddings for formulas and functions to enable natural language search.

By systematically enriching the constellation index, the research ensures that the fire-soot system is not just another piece of code, but a fully documented, queryable, and analyzable entity. This investment in the knowledgebase is what ultimately enables the maximization of AI-chat functionality and supports the long-term vision of a self-documenting, AI-manageable game development platform.

Maximizing AI-Chat Affordances for Design-Time and Post-Hoc Analysis

The successful integration of the fire and soot system hinges on maximizing its utility through AI-chat affordances. The user's clarification emphasizes a strategic focus on design-time support and post-hoc analysis, with runtime assistance as a longer-term goal . This prioritization aligns with the current capabilities of the GameNext architecture and offers the highest immediate leverage for improving developer productivity and creative iteration. The system is designed to be a template for how AI should interact with the GameNext ecosystem: by providing structured, schema-backed access to code, telemetry, and ML models .

Design-time support is where the AI's value is most apparent in the near term. The cornerstone of this capability is the ability to programmatically generate clear, human-readable reports on any given fire-soot profile. A function like `generate_fire_soot_report(profile_id)` would pull information from the

`constellationindex.db`, synthesizing details about the accumulation formulas, cleaning curves, and typical zone configurations into a structured document . This serves as an instant documentation generator, reducing the friction of onboarding new team members and helping experienced designers quickly recall the properties of various profiles.

Beyond simple reporting, the AI can act as a design companion by proposing new configurations and analyzing trade-offs. Because the accumulation and cleaning behaviors are defined as formal formulas in the catalog, the AI can be prompted to manipulate them. For example, a designer could ask, "What if we increase the soot accumulation rate β by 20%? How would that affect the visual result?" The AI could then retrieve the relevant formula, suggest the modification, and potentially generate a visualization or a textual description of the expected outcome. This moves the design process from a manual, trial-and-error loop to an interactive exploration of the system's parameter space. The AI can also help adhere to established constraints, such as ensuring that soot coverage never exceeds a certain threshold (e.g., 0.7 per region) to prevent characters from turning into uniform black blobs, by checking proposed changes against linting rules derived from the formula catalog .

Post-hoc analysis represents the second pillar of AI utility, transforming raw telemetry into actionable insights. With soot and exposure metrics logged into the SQLite database alongside existing BCI data, the AI can answer complex analytical questions that would be tedious or impossible to address manually . Instead of scanning through hours of logged data, a researcher can pose a natural language query like, "Find all gameplay sessions where players spent more than 5 seconds in the inner fire zone and had soot levels exceed 0.8 on their torso" . The AI would translate this query into an optimized SQL statement, execute it against the `constellationindex.db`, and return the relevant data segments or aggregated statistics. This capability is crucial for validating the core hypotheses of the project, such as the link between soot accumulation and player comfort or stress .

This analytical power is further amplified by the integration with machine learning pipelines. The same telemetry export patterns used for BCI data can be applied to the fire/soot metrics . This CSV-exported data, containing time-series data of soot levels, exposure states, and corresponding BCI metrics (like Cortical Intensity Change (CIC) or Arousal Response Rating (ARR)), can be fed into training jobs [20 57](#) . Researchers can train simple regression models to quantify the relationship between visual stimuli (soot buildup) and physiological responses (player arousal). For instance, a model could determine if a particular soot accumulation curve correlates strongly with negative comfort metrics, providing empirical evidence to tune the system for better player

experience. This co-tuning of visual parameters with neuro-response data is a key goal for advancing the realism and emotional impact of games [22](#).

The following table illustrates the progression of AI-chat affordances, from basic to advanced, and their alignment with the research goals:

Affordance Level	Capability	Enabling Technologies / Components	Primary Goal
Level 1: Reporting	Generate structured reports for fire-soot profiles.	Patternized doc generation, constellationindex.db schema extensions.	Lower cognitive load, improve onboarding.
Level 2: Synthesis & Editing	Propose new profiles, tweak formulas, and check constraints.	Formula catalog with symbolic representations, pattern synthesis tools.	Accelerate creative iteration, ensure adherence to design principles.
Level 3: Query & Analysis	Answer natural language questions about historical telemetry.	Semantic search over indexed telemetry, automated SQL query generation.	Enable rapid, data-driven decision making.
Level 4: ML Co-Tuning	Correlate soot accumulation with BCI metrics to optimize for player comfort.	Telemetry export to CSV, integration with offline ML training pipelines.	Quantitatively validate and refine visual design choices.

By focusing initially on these design-time and post-hoc capabilities, the research builds a robust foundation for AI integration. These functionalities provide immediate value to the development team, demonstrate the power of the schema-first approach, and lay the groundwork for more ambitious runtime applications in the future, such as an interactive in-editor explorer that provides real-time feedback on parameter changes .

Engine-Specific Implementation Roadmap and Performance Validation

While the canonical runtime module provides the universal logic, practical implementation requires adapting this design to the unique architectures of target engines like Unreal and Unity. The roadmap is structured as a phased rollout, beginning with the canonical specifications and progressing to thin, engine-specific "adapter" layers that bind the core logic to engine-native systems . This approach ensures that the bulk of the innovation and maintenance effort is concentrated in a single, engine-agnostic location, while still allowing for optimal integration with engine-specific features for materials, particle systems, and scripting.

For Unreal Engine, the implementation would center around creating a custom component, `UFireExposureComponent`, which encapsulates the `FireExposureState` struct. This component would integrate with Unreal's existing damage and overlap notification systems. Instead of creating a new proximity-checking mechanism, it would subscribe to existing damage events generated by `PointDamageEvent` or `RadialDamageEvent` to determine when a character enters or exits a fire zone. This directly addresses the hypothesis that reusing existing damage queries can drive grime updates without extra CPU cost. For the visual aspects, the system would leverage Unreal's `MaterialParameterCollection` (MPC) assets to update material parameters on character meshes [51](#). Alternatively, for more dynamic effects like live-fire flicker or heat haze, a Niagara system could be employed. A key challenge in older Unreal versions was the limited ability to write particle simulation data back to material instances. However, recent developments, including custom Niagara Data Interfaces (NDIs) that can write to dynamic material bindings, provide a robust solution for this, even circumventing limitations related to Nanite support [34](#) [46](#). As Unreal Engine continues to evolve, with recent versions like 5.7 offering significant performance improvements in areas like shadow casting and foliage rendering, the baseline performance of the overall engine will continue to rise, making the addition of this low-cost system increasingly feasible [9](#) [24](#) [30](#).

In Unity, the implementation would follow a similar pattern but utilize Unity's specific paradigms. A `FireExposureBehaviour` `MonoBehaviour` would serve as the container for the `FireExposureState` and would be responsible for performing the necessary physics or trigger checks to sample fire zones each frame. To achieve the high-performance, parallel scalar updates required by the canonical model, this component would work in conjunction with Unity's Job System and the Burst compiler [32](#) [68](#). The Job System is designed to efficiently leverage multi-core CPUs by scheduling work on a thread pool, and Burst compiles C# code to highly optimized native code, capable of delivering massive performance gains over standard managed execution [68](#). The logic for updating the `FireExposureState` across all characters in a scene would be offloaded to a job, ensuring it does not block the main thread. For applying the resulting soot values to materials, the system would use `MaterialPropertyBlock` [42](#) [44](#). This Unity feature is specifically designed to modify material properties on a per-renderer basis without creating new material instances, which is crucial for preserving GPU draw call batching and maintaining performance [69](#). However, developers must be aware that `MaterialPropertyBlock` can break batching for non-instanced shaders, meaning the target character shaders must be written to support GPU Instancing for the full performance benefit to be realized [69](#) [70](#).

Regardless of the engine, rigorous performance validation is paramount to ensure the system adheres to its "near-zero cost" promise. The validation process must instrument the `fire_soot` module as a distinct pattern within the existing energy profiling tools . Key metrics to measure and compare against baseline (no fire/soot system) include:

- **CPU Cost:** The average time taken per frame to execute the ``update_fire_soot_state`` function across all active characters. This should register as a negligible amount, ideally below 0.1 milliseconds even with hundreds of characters.
- **Cache Impact:** The effect on CPU cache performance. The scalar, contiguous memory access pattern of the update logic is expected to be highly cache-friendly, but this should be verified using profiling tools.
- **GPU Overhead:** A definitive measurement showing zero additional render targets, no new texture allocations, and no increase in draw calls. This is a core requirement of the design .

The following table provides a comparative overview of the implementation strategy for Unreal and Unity:

Aspect	Unreal Engine Implementation	Unity Implementation
Core Logic Container	UFireExposureComponent (C++)	FireExposureBehaviour (C#, Jobified) 68
Proximity Sampling	Reuse existing damage/radial damage events .	Physics.OverlapSphere/Trigger checks or custom distance queries 52 .
Visual Parameter Update	UMaterialParameterCollection or Niagara Data Interfaces (NDIs) 34 51 .	MaterialPropertyBlock 42 69 .
High-Performance Updates	Not explicitly mentioned, but C++ logic is inherently performant.	Utilize Unity's Job System and Burst compiler for parallel, optimized execution 32 68 .
Key Considerations	Leverage existing damage systems; NDIs solve material writing issues.	Ensure shaders support GPU Instancing to avoid batching breaks when using MaterialPropertyBlock 69 .

By following this detailed roadmap and subjecting the implementation to strict performance validation, the research can confidently deliver a system that meets its core objectives of being performant, portable, and seamlessly integrated into the leading game development engines.

Strategic Research Tracks and Validation Framework

To guide the development and ensure the success of the fire and soot initiative, the research must be framed within a series of concrete, testable research objects and hypotheses. These objects, proposed as new entries in the GameNext playbook, provide a structured framework for experimentation, data collection, and analysis . Each object is anchored in a specific domain and is designed to validate a key assumption of the overall system, from its rendering fidelity to its cross-cutting impact on player experience.

The first research object, **fire-soot-accumulation-model**, resides in the domain of rendering. Its central hypothesis is that a per-region soot accumulation model driven by zone-based exposure can produce believable fire damage visuals at a constant, near-zero frame cost . The primary research task is to conduct a systematic parameter sweep over the core coefficients: k_{in} (heat/smoke build-up rate), k_{out} (decay rate), α (smoke contribution to soot), and β (heat contribution to soot) . A dedicated Python or Rust simulation tool can be developed to model a character's exposure trajectory over time, logging the resulting soot accumulation curves to CSV files. This allows for rapid tuning and visualization of different parameter sets to find perceptually stable regimes that feel responsive near flames but accumulate smoothly with repeated exposure. A secondary task is to integrate these exposure and soot trajectories with BCI metrics collected during playtests, training simple ML models to correlate the visual build-up patterns with player comfort metrics like CDL (Comfort Discomfort Likert scale) or ARR (Arousal Response Rating) to ensure the effect feels satisfying rather than annoying [20](#) .

The second research object, **fire-zone-damage-and-grime-coupling**, falls under the physics/rendering domain. Its hypothesis posits that it is possible to reuse existing engine damage radius queries to drive grime updates without incurring any new proximity-checking costs . The research task here involves instrumentation. The existing damage query systems in Unreal and Unity must be modified to output not just a damage value, but also a "zone occupancy" signal (e.g., inside inner zone, mid zone, outer zone) . The grime update logic is then attached to the same processing step that handles the damage calculation. The validation involves precise measurement of the CPU cost and cache impact of this coupled system compared to running separate fire-soot and damage systems. The results of this measurement would be logged in a `patternenergyprofiles` table, contributing to the constellation's growing library of performance data for different visual patterns .

The third research object, **fire-cleaning-mechanics-visual-contract**, is a cross-cutting study focused on player experience. Its hypothesis suggests that linking the

removal of soot to established healing or cleaning UX (e.g., visiting a med-center, taking a shower) helps maintain the effect's believability while preventing it from becoming a source of annoyance . The research tasks involve conducting controlled A/B tests on different cleaning mechanics. Variables to test include the type of cleaning event (med-bay visit vs. walking through rain vs. a scripted interaction), the cleanup rate, and the total duration of the cleaning animation. The effectiveness of each mechanic can be measured by analyzing telemetry data exported via the existing tooling to see how quickly soot is removed and how players react to the visual and audio feedback during the cleaning sequence . Based on these findings, the research aims to define a set of standard "clean arcs" for the GameNext UX toolkit, providing designers with proven, out-of-the-box solutions for managing character cleanliness .

The following table summarizes the proposed research tracks:

Research Object ID	Domain	Core Hypothesis	Key Research Tasks & Validation Methods
fire-soot-accumulation-model	Rendering	Zone-based, per-region soot accumulation produces believable damage at constant frame cost.	Parameter sweep of integrator coefficients (k_{in} , k_{out} , α , β). Simulate exposure paths. Correlate soot trajectories with BCI comfort metrics via ML models.
fire-zone-damage-and-grime-coupling	Physics/ Rendering	Reusing existing damage queries to drive grime updates incurs no additional proximity-checking cost.	Instrument damage queries to output zone signals. Attach grime updates to the same engine tick. Measure and log CPU cost and cache impact in <code>patternenergyprofiles</code> .
fire-cleaning-mechanics-visual-contract	Cross-Cutting (UX)	Tying grime removal to healing/cleaning UX maintains believability while avoiding annoyance.	A/B test different cleaning event types (med-bay, rain) and cleanup rates. Analyze telemetry to measure cleanup speed and player reaction. Define standard "clean arcs" for the UX toolkit.

Collectively, these three research tracks form a comprehensive validation framework. They move the project from a theoretical design to a scientifically-grounded system, producing empirical data that can inform not only the final implementation of fire and soot but also the future evolution of the entire GameNext platform. By establishing clear hypotheses and measurable outcomes, the research ensures that every design decision is backed by evidence, ultimately leading to a more effective and impactful tool for game developers.

Reference

1. A Connectionist Encoding of Schemas and Reactive Plans https://www.researchgate.net/publication/2469565_A_Connectionist_Encoding_of_Schemas_and_Reactive_Plans
2. (PDF) "Design at the Border: Liminality, the Virtual, and Interior ... https://www.academia.edu/98919070/_Design_at_the_Border_Liminality_the_Virtual_and_Interior_Transformation_from_Antiquity_to_Mixed_Reality_
3. Sensors, Volume 26, Issue 4 (February-2 2026) – 321 articles - MDPI <https://www.mdpi.com/1424-8220/26/4>
4. Appl. Sci., Volume 13, Issue 19 (October-1 2023) – 562 articles - MDPI <https://www.mdpi.com/2076-3417/13/19?qZ6l35NMygkD9=pjq4rD2BRWmjbC>
5. Sensors, Volume 20, Issue 1 (January-1 2020) – 325 articles <https://www.mdpi.com/1424-8220/20/1>
6. MIT Manipal Academic Handbook 2025-26 | PDF - Scribd <https://www.scribd.com/document/892926640/Academic-Handbook-2025-26-MIT-Manipal-2-250711-223754-1>
7. words-333333 - cs.Princeton <https://www.cs.princeton.edu/courses/archive/spring18/cos226/assignments/autocomplete/testing/words-333333.txt>
8. Ultimate list of popular, gaming-related terms - PlayStation <https://www.playstation.com/en-us/editorial/playstation-ultimate-gaming-glossary/>
9. Unreal Engine 5.7 Release Notes - Epic Games Developers <https://dev.epicgames.com/documentation/unreal-engine/unreal-engine-5-7-release-notes>
10. Unreal Engine 5.7 is now available <https://www.unrealengine.com/news/unreal-engine-5-7-is-now-available>
11. Changes of parameters from materials do not update in viewport ... <https://forums.unrealengine.com/t/changes-of-parameters-from-materials-do-not-update-in-viewport-after-ue-restart-until-recompiling/417744>
12. Unreal Engine 5.0 Release Notes - Epic Games Developers https://dev.epicgames.com/documentation/en-us/unreal-engine/unreal-engine-5.0-release-notes?application_version=5.0
13. Unreal Engine Console Variables Reference <https://dev.epicgames.com/documentation/unreal-engine/unreal-engine-console-variables-reference?lang=ja>

14. Unreal Engine 4.26.1(windows) Console Variables and Commands <https://www.cnblogs.com/kekec/p/16872555.html>
15. 2. Building the Firework Head in Unreal Editor for Fortnite <https://dev.epicgames.com/documentation/fortnite/2-building-the-firework-head-in-unreal-editor-for-fortnite>
16. impact of image quality on measures of bone density and ... <https://pubmed.ncbi.nlm.nih.gov/22019605/>
17. glove.6B.100d.txt-vocab.txt - CodaLab Worksheets <https://worksheets.codalab.org/rest/bundles/0xadf98bb30a99476ab56ebff3e462d4fa/contents/blob/glove.6B.100d.txt-vocab.txt>
18. [2206.11380] Positional Paper: Schema-First Application Telemetry <https://arxiv.org/abs/2206.11380>
19. [PDF] Positional Paper: Schema-First Application Telemetry - arXiv <https://arxiv.org/pdf/2206.11380>
20. Research on a Visual Comfort Model Based on Individual ... - MDPI <https://www.mdpi.com/2071-1050/13/14/7602>
21. STRUCT::Home <https://www.icst.pku.edu.cn/struct/>
22. TDP-DETR: Temporal dynamics perception framework for video ... <https://pubmed.ncbi.nlm.nih.gov/42102552/>
23. UE 5.5 bad performance - Page 2 - Rendering <https://forums.unrealengine.com/t/ue-5-5-bad-performance/2220019?page=2>
24. Tom Looman - Unreal Engine 5.7 Performance Highlights - LinkedIn https://www.linkedin.com/posts/tomlooman_unreal-engine-57-performance-highlights-activity-7399126695623942144-iqX2
25. English.txt - APRIL robotics lab https://april.eecs.umich.edu/courses/eecs492_w11/wiki/images/8/82/English.txt
26. Unreal Engine 5.0 Release Notes https://dev.epicgames.com/documentation/unreal-engine/unreal-engine-5.0-release-notes?application_version=5.0&lgNYsuL=UjWabWGy1Cn
27. All the big news from the State of Unreal at GDC '24 <https://www.unrealengine.com/blog/all-the-big-news-from-the-state-of-unreal-at-gdc-24>
28. A roadmap of Unity Engine and services | GDC 2024 <https://discussions.unity.com/t/unity-6-and-beyond-a-roadmap-of-unity-engine-and-services-gdc-2024/942714?page=2>
29. GDC State of Unreal 2024 <https://gamefromscratch.com/gdc-state-of-unreal-2024/>

30. Unreal Engine 5.7: A Step Forward for Large-Scale Projects https://www.linkedin.com/posts/matteo-maniscalco_unreal-engine-57-is-officially-released-activity-7396181598406356993-QaDG
31. Cognitive Development in Children | PDF | Thought - Scribd <https://www.scribd.com/document/732008860/John-H-Flavell-Patricia-H-Miller-Scott-a-Miller-Cognitive-Development-Prentice-Hall-2002>
32. Burst User Guide | Burst | 1.3.9 - Unity - Manual <https://docs.unity3d.com/Packages/com.unity.burst@1.3/manual/index.html>
33. Burst User Guide | Burst | 1.2.3 - Unity - Manual <https://docs.unity3d.com/Packages/com.unity.burst@1.2/manual/index.html>
34. Pull Request: Niagara Data Interface - Material Instance Dynamic <https://forums.unrealengine.com/t/pull-request-niagara-data-interface-material-instance-dynamic/2552586>
35. Niagara | Unreal Engine 5.7 Documentation - Epic Games Developers <https://dev.epicgames.com/documentation/unreal-engine/API/Plugins/Niagara>
36. Game Engine Architecture Samples - LinkedIn <https://www.linkedin.com/pulse/game-engine-architecture-samples-christian-sagel>
37. Game engine architecture - c++ - Stack Overflow <https://stackoverflow.com/questions/79892947/game-engine-architecture>
38. GAMES104 Lecture 02 Layered Architecture of Game Engine - 博客园 <https://www.cnblogs.com/Tellulu/p/18391251>
39. A Taxonomy of Game Engines and the Tools that Drive the Industry https://www.researchgate.net/publication/337165844_A_Taxonomy_of_Game_Engines_and_the_Tools_that_Drive_the_Industry
40. sqlalchemy/sql : cross model relationships, getting info from 'cousin ... <https://stackoverflow.com/questions/12477437/sqlalchemy-sql-cross-model-relationships-getting-info-from-cousin-models>
41. Material Property Block (803836) - Unity Discussions <https://discussions.unity.com/t/material-property-block-803836/803836>
42. Scripting API: MaterialPropertyBlock - Unity - Manual <https://docs.unity3d.com/ScriptReference/MaterialPropertyBlock.html>
43. Unity渲染优化的4种批处理- SRP Batchers 与GPU Instancing - 知乎专栏 <https://zhuanlan.zhihu.com/p/432223843>
44. 简单了解一下Unity的MaterialPropertyBlock-CSDN博客 <https://blog.csdn.net/ZeroBugX/article/details/146689496>
45. What is a viable substitute for MaterialPropertyBlocks in HDRP? <https://discussions.unity.com/t/what-is-a-viable-substitute-for-materialpropertyblocks-in-hdrp/859058>

46. Niagara Data Interfaces | Epic Developer Community <https://dev.epicgames.com/community/learning/knowledge-base/jPMm/unreal-engine-niagara-data-interfaces>
47. Shankland - Theoretical Rook Endgames (2023) | PDF - Scribd <https://www.scribd.com/document/729012682/Shankland-Theoretical-Rook-Endgames-2023>
48. Generative AI in Dynamic Chat | Adobe Marketo Engage <https://experienceleague.adobe.com/en/docs/marketo/using/product-docs/demand-generation/dynamic-chat/generative-ai/overview>
49. AI Chatbot Design Best Practices for Website Owners | Nick Babich ... https://www.linkedin.com/posts/nbabich_the-right-way-to-create-ai-chatbots-ai-activity-7447222040148815872-Y3hK
50. Video: Build a RAG-Powered Chatbot in Five Minutes <https://developer.nvidia.com/blog/video-build-a-rag-powered-chatbot-in-five-minutes/>
51. Using Material Parameter Collections in Unreal Engine <https://dev.epicgames.com/documentation/unreal-engine/using-material-parameter-collections-in-unreal-engine>
52. Entity Component System - an old new thing - DEV Community <https://dev.to/kspeakman/entity-component-system-an-old-new-thing-3224>
53. MaterialPropertyBlock - Unity 脚本API <https://docs.unity3d.com/cn/2020.2/ScriptReference/MaterialPropertyBlock.html>
54. Unity常见合批失败 - 知乎专栏 <https://zhuanlan.zhihu.com/p/109200416>
55. Unreal Engine Leads GDC State of Industry Report with 42% Adoption https://www.linkedin.com/posts/chris-gardner-42b3b672_gdc-2026-state-of-the-game-industry-reveals-activity-7445409001460133888-Pl5l
56. GDC 2026 - Unreal Engine <https://www.unrealengine.com/events/gdc-2026>
57. Study of Human Visual Comfort Based on Sudden Vertical ... - MDPI <https://www.mdpi.com/2075-5309/12/8/1127>
58. (PDF) An occupant-centric approach for spatio-temporal visual ... https://www.researchgate.net/publication/372196310_An_occupant-centric_approach_for_spatio-temporal_visual_comfort_assessment_and_optimization_in_daylit_sports_spaces
59. Impact of lighting environment on human performance and ... <https://pubmed.ncbi.nlm.nih.gov/38552981/>
60. ACCV 2022 papers - CVF Open Access https://openaccess.thecvf.com/content/ACCV2022/html/Hu_Learning_to_Predict-Decomposed_Dynamic_Filters_for_Single_Image_Motion_AcCV_2022_paper.html
61. 详情 - 图书馆书目检索系统 <https://opac.lib.dlut.edu.cn/mspace/searchDetailLocal/m63cd32200082181af178a705cfc8a771>

62. A Hybrid Game Engine–Generative AI Framework for Overcoming ... <https://www.mdpi.com/2504-4990/8/4/99>
63. Ernie 4.0 token使用教材+ 图Rag检索生成 - 飞桨AI Studio - 百度 <https://aistudio.baidu.com/projectdetail/8584465>
64. Particle Update Group Reference for Niagara Effects in Unreal Engine <https://dev.epicgames.com/documentation/unreal-engine/particle-update-group-reference-for-niagara-effects-in-unreal-engine>
65. Unreal Engine Issues and Bug Tracker (UE-229949) <https://issues.unrealengine.com/issue/UE-229949>
66. Particle Spawn Group Reference for Niagara Effects in Unreal Engine <https://dev.epicgames.com/documentation/unreal-engine/particle-spawn-group-reference-for-niagara-effects-in-unreal-engine>
67. Unreal Engine Issues and Bug Tracker (Search Results) <https://issues.unrealengine.com/issue/search?q=&resolution=&type=&project=&sort=resolution&page=221>
68. Unity JobSystem使用及技巧- 飞翔的子明 - 博客园 <https://www.cnblogs.com/FlyingZiming/p/17241013.html>
69. Unity_同个材质多个颜色内存优化——使用MaterialPropertyBlock 原创 <https://blog.csdn.net/avi9111/article/details/125862150>
70. Learn how to use MaterialPropertyBlocks and [PerRendererData] for ... <https://discussions.unity.com/t/learn-how-to-use-materialpropertyblocks-and-perrendererddata-for-great-performance-gains/628464>